IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 29 June 2023, accepted 11 August 2023, date of publication 15 August 2023, date of current version 29 August 2023.

Digital Object Identifier 10.1109/ACCESS.2023.3305375

RESEARCH ARTICLE

Joint Reputation Based Grouping and Hierarchical Byzantine Fault Tolerance Consensus Protocol

HAO QIN[1] AND YEPENG GUAN[1,2,3]

[1]School of Communication and Information Engineering, Shanghai University, Shanghai 200444, China
[2]Key Laboratory of Advanced Display and System Application, Ministry of Education, Shanghai 200072, China
[3]Key Laboratory of Silicate Cultural Relics Conservation (Shanghai University), Ministry of Education, Shanghai 200444, China

Corresponding author: Yepeng Guan (ypguan@shu.edu.cn)

This work was supported in part by the National Key Research and Development Program of China under Grant 2020YFC1523004.

ABSTRACT Consensus protocol has challenges in terms of low consensus efficiency and centralization, as well as poor fault tolerance. A joint reputation model based grouping and hierarchical byzantine fault tolerance consensus protocol has been proposed. It is composed of both grouping and hierarchical models. The grouping model uses joint reputation values to balance node grouping, minimize overall differences in joint reputation values among groups. Some nodes with a joint reputation value ranking in the top 50% of the group are randomly selected as function ones, which improves the degree of decentralization compared to some election strategies of leader nodes. In the hierarchical model, nodes in each group are layered and the supervision nodes are mapped to the upper layer, which improves consensus network security and efficiency. Besides a new communication structure has been designed to improve fault tolerance and reduce communication complexity by improving the three phases of Practical Byzantine Fault Tolerant (PBFT). Comparative experiments have shown the superiority of the developed protocol over other existing protocols.

INDEX TERMS Blockchain, fault tolerance, consensus protocol, reputation model, distributed networks.

I. INTRODUCTION

In recent years, blockchain technology has been widely studied with the rapid development of Bitcoin [1], [2]. Blockchain is a distributed network system, in which each node maintains an append-only ledger [3], [4]. The system has the characteristics of decentralization, tamper-proof, traceability, and programmability, and holds huge promise for the Internet of Things (IoT), finance, logistics, and health-care [5], [6]. Blockchain is generally classified into three types based on the degree of decentralization, namely public blockchain, consortium blockchain, and private blockchain. The public blockchain [7] is a highly decentralized consensus network where any node can join the network at any time. The chain is typically used to accept untrusted and high latency nodes. Consensus protocols for the public chain mainly include Proof-of-Work (PoW) [8], Proof-of-Stake (PoS) [9], and Delegated Proof-of-Stake (DPoS) [10]. The consortium chain [11], [12] is a consensus network with a partial degree of decentralization. Nodes joining the network are usually managed and shared by several institutions. The private chain [13], [14] is a highly centralized consensus network where nodes are usually controlled by one institution or individual.

Consensus protocol serves as the core technology for blockchain networks [15], [16], which largely determines the performance of the network system, such as throughput, fault tolerance, efficiency, scalability, and so on. Practical Byzantine Fault Tolerance (PBFT) consensus protocol was proposed to protect consortium blockchain systems in [17]. The protocol is an improved and practical protocol based on the original Byzantine Fault Tolerance (BFT) [18]. PBFT [17] reduces the complexity of BFT [18] from exponential to polynomial. However, PBFT [17] has problems with poor scalability, high communication complexity and low fault tolerance. Therefore, PBFT [17] is generally only applicable to the consensus network with less than 100 network nodes, which is challenging to use in a wider network [19], [20].

The associate editor coordinating the review of this manuscript and approving it for publication was Mueen Uddin.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

In recent years, some BFT-based consensus protocols have been proposed to address PBFT [17] scalability issues. A multi-layer consensus protocol [21] was developed to reduce communication complexity. It assumes that fault nodes only exist at the bottom layer, while nodes at other layers are normal. However, the protocol cannot be applied to a real consortium blockchain [22]. A Delegate Byzantine Fault Tolerance consensus protocol (DBFT) [23] splits nodes into multiple clusters. Each cluster selects a delegated node to represent the cluster, which can reduce communication complexity by exchanging confirmed information between representative nodes. However, the protocol cannot tolerate that the delegate node is a Byzantine node [24]. A hash-ring based consensus protocol (HC-PBFT) [25] was designed to reduce communication complexity, which used hierarchical technology to avoid direct communication between a large number of nodes. But its upper nodes still communicate directly, which makes it difficult to achieve high fault tolerance.

Some grouping methods are proposed to improve consensus efficiency. A K-medoids-based approach was proposed to reach consensus within groups [26]. A network was designed to group nodes according to their communication capabilities [27]. Nodes are grouped according to their geographical locations [28]. NBFT [29] was proposed to avoid much communication between nodes by using the consistent hash algorithm to group consensus nodes. A score-based consensus protocol (SG-PBFT) [30] was designed to group nodes. However, these grouping methods [26], [27], [28], [29], [30] do not consider the distribution of different performance or behavioral reputation nodes in a comprehensive manner, which results in significant differences between groups. These differences can affect the effectiveness of grouping strategies and lead to nodes in a particular group without expected speed and security of consensus.

Some methods based on reputation have been proposed to improve node reliability. T-PBFT [31] evaluates node trust by the transactions between nodes. A reputation based PBFT was introduced in [32], which could enhance node reliability through penalty mechanisms. A protocol [33] was designed to obtain a node reliability rating by dynamically evaluating the real-time performance of the service. These works [31], [32], [33] only consider node performance reputation or node behavior reputation, so they perform poorly in evaluating node reliability.

Digital signature technology was utilized to aggregate vote results from the nodes into the primary node [34]. The SBFT [35] sets up a collector that assembles and forwards voting data from each node throughout the consensus process. However, these protocols [34], [35] typically allocate more time to decryption and encryption, and their security has yet to be formally assessed.

A Joint Reputation based Grouping and Hierarchical Byzantine Fault Tolerance consensus protocol (JR-GHBFT) has been proposed to address the above issues in this paper. A joint reputation model has devised to solve the problem of poor node reliability evaluation in some methods [31], [32], [33]. A grouping model is developed to minimize the overall differences in node distribution between groups for certain methods [26], [27], [28], [29], [30]. An election strategy has been proposed to improve the degree of decentralization compared to some election strategies [31], [32], [33] of leader nodes. A hierarchical model is built to improve consensus network security in contrast to HC-PBFT [25]. Moreover, a new data transmission structure has been designed to reduce PBFT [17] communication complexity and improve HC-PBFT [25] fault tolerance. This new structure can be applied to improve SBFT [35] scalability. Some main contributions are as follows.

1) The joint reputation model has been proposed to improve node reliability. The credibility of each node based on node behavior and performance can be evaluated in this model. The model can also be applied to differentiate between normal and Byzantine nodes.
2) The joint reputation based grouping and hierarchical model has been proposed to improve consensus efficiency. It is composed of both grouping and hierarchical models. The grouping model uses joint reputation values to balance node grouping, and minimize overall differences in joint reputation values among groups. Some nodes with a joint reputation ranking in the top 50% of this group are randomly selected as function nodes to improve the degree of decentralization. In the hierarchical model, nodes in each group are layered and the supervision nodes are mapped to the upper layer, which improves consensus network security and efficiency.
3) The new data transmission structure has been designed to improve fault tolerance and reduce communication complexity.

Comparative experiments have shown the superiority of the developed protocol over other existing protocols.

The remainder of the paper is arranged as follows. A joint reputation model is illustrated in Section II. Joint reputation based grouping and hierarchical model is illustrated in Section III. JR-GHBFT consensus protocol is described in Section IV. Section V discusses performance analysis. Section VI describes the experimental results and discussions, while Section VII provides some conclusions.

II. JOINT REPUTATION MODEL

In a consensus network, nodes are typically made up of both normal and Byzantine nodes. Some consensus algorithms, including PBFT [17] and DBFT [23], are unable to differentiate between normal nodes and Byzantine nodes before and after the consensus process. Some reputation-based methods [31], [32], [33] have been proposed to differentiate between normal and Byzantium nodes. These works [31], [32], [33] only consider node performance reputation or node behavior reputation, so they perform poorly in evaluating node reliability.

A joint reputation model is designed to overcome the limitations mentioned above. It is composed of node

performance reputation model and node behavior reputation model. Firstly, a node performance reputation model is described in part A of this section, which is used to measure the network performance of nodes. In part B of this section, a node behavior reputation model is proposed to evaluate the behavior of nodes throughout the consensus process. Some details are described as following.

A. NODE PERFORMANCE REPUTATION MODEL

Nodes could usually be considered as providers in the blockchain network. Response delay is an indicator to measure the network performance of the node. If the node has high data throughput and low response delay, it would have high performance and reliability. The calculation of response delay is as following.

For the case where node n serves node k, the response delay of node n can be defined as the difference between the start time of node k message transmission and the end time when node n completes processing node k information. When node n serves multiple nodes, the response delay of node n can be taken as the average of the response delays of all nodes it serves.

To get the average response delay of all nodes, the average network response delay is defined in a cycle as follows:

$$\overline{T_{aveN}} = \frac{1}{N}\sum_{n=1}^{N}\overline{T_n} \tag{1}$$

where N denotes the total number of nodes in the network, $\overline{T_n}$ represents average response delay of node n serving other nodes in a round.

To determine the superiority or inferiority of the average response delay of node j relative to other nodes, relative average response delay of node j serving other nodes is then defined in a cycle as follows:

$$\overline{T_{reAve}} = \frac{1}{X}\sum_{j=1}^{X}\left(\overline{T_{aveN}} - \overline{T_j}\right) \tag{2}$$

where X is the total number of cycles that node j serves for other nodes, $\overline{T_j}$ represents average response delay of node j serving other nodes in a cycle.

Relative average response time can effectively reflect node performance relative to other nodes. The larger the relative average response delay, the better the reliability and performance of the network node. To normalize the relative average response delay and better reward and punishment nodes, we define a reward and punishment function as follows:

$$R_{pRP} = \frac{2}{\pi}\tan^{-1}\left(\overline{T_{reAve}} \times w\right) \tag{3}$$

where $\times$ is a multiple operator, w is the weight which is used to balance the normalized reward and punishment function and set to 1000 in the experiment. Relative average response delay of the node can be more effectively normalized in $\tan^{-1}()$ function.

A developed reward and punishment strategy is different from previous strategies. Firstly, the performance reputation value of a node is dynamically calculated based on the average network response delay. As the performance of network nodes becomes better, the average network response delay would decrease, which would urge nodes to provide better services in order to obtain a higher performance reputation. Secondly, it is easy to calculate the performance reputation of nodes, thereby reducing computational costs.

The node performance reputation value would be updated after this cycle as follows:

$$P^n = P^{n-1} + R_{pRP} \times P^{n-1} \tag{4}$$

where P^n denotes the node performance reputation value in the nth round. The initial performance reputation value P^0 is zero, where P^0 denotes the node performance reputation value in initial state.

B. NODE BEHAVIOR REPUTATION MODEL

Performance reputation can ensure the reliability of the node to a certain extent. However, it cannot measure malicious behavior of the nodes with high performance reputation values during consensus. A node behavior reputation model has been proposed to evaluate node behavior during consensus as following.

The valid response is introduced as a quantitative index to evaluate node behavior. We judge whether the response of node n is valid through other nodes. $(N_n - M_n)$ has been used to measure whether the node n has valid network response or not. Judgment rules involve detecting whether the received message has been tampered and whether the message has timed out. When a node among other nodes confirms that the response of node n is normal, the value of N_n (representing the number of normal responses) for node n increases by 1. Conversely, if the response is considered abnormal, the value of M_n (representing the number of abnormal responses) for node n increases by 1.

To get the average number of valid responses of all nodes, the average number of valid network responses is defined in a cycle as follows:

$$\overline{C_{aveN}} = \frac{1}{N}\sum_{n=1}^{N}\sum_{l=1}^{X}\left(N_n^l - M_n^l\right) \tag{5}$$

where N_n^l denotes whether node n is a normal response in the lth consensus. The value of N_n^l is 0 or 1, and $N_n^l = 1$ denotes a normal response. M_n^l represents whether node n is a mistaken response in the lth consensus. The value of M_n^l is 0 or 1, and $M_n^l = 1$ represents a mistaken response.

To determine the superiority or inferiority of the response of node j compared to other nodes, relative average number of valid responses is then defined in a cycle as follows:

$$\overline{C_{reAve}} = \sum_{l=1}^{X}\left(N_j^l - M_j^l\right) - \overline{C_{aveN}} \tag{6}$$

where N_j^l denotes whether node j is a normal response in the lth consensus. The value of N_j^l is 0 or 1, and $N_j^l = 1$ denotes a

normal response. M_j^l represents whether node j is a mistaken response in the lth consensus. The value of M_j^l is 0 or 1, and $M_j^l = 1$ represents a mistaken response.

Relative average number of valid responses can effectively reflect the degree of superiority or inferiority of node behavior relative to other nodes. The larger the relative average number of valid responses, the better the reliability of the network node. To normalize the relative average number of valid responses and better reward and punishment nodes, we define a reward and punishment function as follows:

$$R_{bRP} = \frac{2}{\pi} \tan^{-1}\left(\overline{C_{reAve}}\right) \tag{7}$$

The node behavior reputation value would be updated after this cycle as follows:

$$B^n = B^{n-1} + R_{bRP} \times B^{n-1} \tag{8}$$

where B^n denotes the node behavior reputation value in the nth round. The initial behavior reputation value B^0 is zero, where B^0 denotes the node behavior reputation value in initial state.

Since when nodes with high performance reputation values engage in malicious behavior, the current performance reputation update mechanism is not sufficient to effectively restrict their behavior. Therefore, when the relative average number of valid responses is less than a predetermined threshold Th, the performance reputation update function is redefined as follows:

$$P^n = R_{pRP} \times P^{n-1} \tag{9}$$

The Th is defined as follows:

$$Th = \frac{\overline{C_{aveN}} - \overline{C_{pAveN}}}{2} \tag{10}$$

where $\overline{C_{pAveN}}$ demotes the average number of valid network responses for primary node. The joint reputation value of a node can be calculated by combining performance reputation and behavior reputation, as shown below:

$$J = \mu \times P^n + (1 - \mu) \times B^n \tag{11}$$

where μ is the weight of the joint reputation value, which would be discussed later.

Each node in the network possesses a trusted container that is not controlled by itself. The joint reputation value of a node is automatically computed and disseminated through these trusted containers.

In order to better illustrate the relationship between performance reputation and behavior reputation, Algorithm 1 outlines the process of updating joint reputation values in a more intuitive way.

Algorithm 1 Updating Joint Reputation Value

Input:
Consensus cycles, X;
Average network response delay, $\overline{T_{aveN}}$;
Average number of valid network responses, $\overline{C_{aveN}}$;
Weight of joint reputation value, μ;
Output:
R;
for each $j \in [1, X]$ **do**
  $\overline{T_{reAve}} = \overline{T_{reAve}} + \overline{T_{aveN}} - \overline{T_j}$;
  $\overline{C_{reAve}} = \overline{C_{reAve}} + N_j - M_j$;
end for
$\overline{T_{reAve}} = \overline{T_{reAve}}/X$;
$\overline{C_{reAve}} = \overline{C_{reAve}} - \overline{C_{aveN}}$;
$R_{pRP} = \frac{2}{\pi} \tan^{-1}\left(\overline{T_{reAve}} \times w\right)$;
$R_{bRP} = \frac{2}{\pi} \tan^{-1}\left(\overline{C_{reAve}}\right)$;
if $\overline{C_{reAve}} \geq Th$ **then**
  $P^n = P^{n-1} + R_{pRP} \times P^{n-1}$;
else
  $P^n = R_{pRP} \times P^{n-1}$;
end if
$B^n = B^{n-1} + R_{bRP} \times B^{n-1}$;
$R = \mu \times P^n + (1 - \mu) \times B^n$;
return R;

III. JOINT REPUTATION BASED GROUPING AND HIERARCHICAL MODEL

In order to ensure consistency and improve the security and efficiency of consensus networks, a joint reputation based grouping and hierarchical model has been developed. It is composed of both grouping and hierarchical models. Grouping model based on joint reputation is described for grouping nodes in part A of this section. Hierarchical model is proposed in part B of this section, which maps different types of nodes in each group to one of the two layers.

A. GROUPING MODEL

The scalability of PBFT [17] would be limited in large-scale networks when the number of nodes increases due to the lack of grouping nodes in PBFT [17]. Several grouping strategies [26], [27], [28], [29], [30] have been proposed to enhance PBFT [17] scalability in large networks. However, these methods [26], [27], [28], [29], [30] cannot effectively capture the distribution of performance and behavior reputation nodes. It would affect the effectiveness of the grouping policy and cause certain nodes within the group to fail to achieve the expected level of consensus speed and security. A grouping model based on joint reputation is proposed to ensure consistency of overall performance and behavior reputation among groups.

We divide consensus nodes into function nodes and ordinary nodes. Function nodes are divided into four types of nodes with different responsibilities including verification node, collection node, supervision node, and primary node.

The primary node authorizes client requests and collects votes from each group. The verification node verifies authorized client messages. The collection node collects votes for this group. The supervision node supervises other function nodes in this group. The ordinary node is honest and has the opportunity to be selected as function node.

There is only one collection node or primary node in each group. The consensus network has only one primary node in a

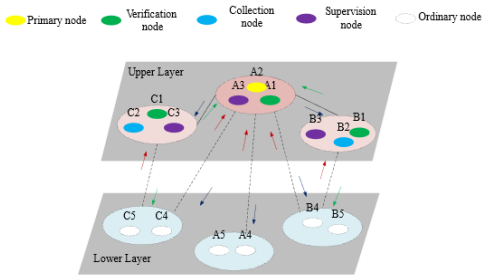


FIGURE 1. The hierarchical structure of JR-GHBFT.

cycle. Some nodes with a joint reputation ranking in the top p of this group have the opportunity to become function nodes. p is the proportion of the number of candidate nodes to the total number of nodes, which would be discussed later. The grouping process is shown in Algorithm 2.

Algorithm 2 Grouping Process

Input:
Node set, *Nodes*;
Joint reputation model, R;
Number of groups, x;
for each $j \in Nodes$ **do**
  Allocate node i to a group in a balanced manner based on joint reputation value;
end for
for each $j \in [1, x]$ **do**
  Sort nodes in group j based on joint reputation value;
  Select randomly function nodes in group j with the top 50% joint reputation values;
end for
Select primary node from collection nodes;

B. HIERARCHICAL MODEL

HC-PBFT [25] maps different types of nodes in each group to one of two layers. However, it has only one node per group mapped to the upper layer, which makes it difficult to ensure the security and efficiency of consensus networks.

A hierarchical model based on joint reputation has been designed to address the above issues. It places three function nodes in each group on the upper layer, while the remaining nodes are placed on the lower layer. There is a supervision node that monitors other function nodes in the group, and the two upper layer nodes divide labor to complete the task of one upper layer node of the original HC-PBFT [25]. The hierarchical structure is shown in Fig. 1.

The number of nodes in each group is approximately the same, which ensures consistency of overall performance and behavior reputation among groups. The relationship between the number of groups and the total number of network nodes will be discussed and analyzed later.

IV. JR-GHBFT CONSENSUS PROTOCOL

PBFT [17] has problems with high communication complexity and low fault tolerance due to the direct communication between nodes. To improve fault tolerance, SBFT [35] collects votes from each node by using aggregate signature technology. It is in a high workload state due to receiving a large number of messages sent by other nodes at the same time. HC-PBFT [25] uses layered technology to avoid direct communication among a large number of nodes for reducing communication complexity. However, its upper layer nodes still communicate directly with each other so that it is difficult to reach high fault tolerance.

A consensus protocol has been designed to address the above issues as shown in Fig. 2. The specific process of JG-GHBFT data transmission is as follows:

Request phase **:** a client initiates a transaction signed by itself with the private key. It then sends the transaction to the primary node.

Pre_prepare phase **:** the primary node determines whether the transaction is legal. If so, it generates a block, and then sends hash input code and *Pre_prepare*1 message to client, other nodes, respectively. Hash input code is composed of client *IP*, random number, and transaction. The *Pre_prepare*1 message format is:

$$< Pre_prepare1, < h, v, S(p), bs, block >, ho > \quad (12)$$

where h is the current block height, v is the current view number. $S(p)$ is the signature of the primary node. bs is the summary of block. ho is the hash output code generated by the primary node.

Hash output code is the result of the hash function calculating the hash input code. The client sends the *Pre_prepare*2 messages to the verification nodes after it is received the hash input code sent by the primary node. The *Pre_prepare*2 message format is:

$$< Pre_prepare2, hi, m > \quad (13)$$

where hi is hash input code generated by the primary node. m is a transaction message encrypted by the private key of client.

The verification nodes will determine whether the block contains the m after it received *Pre_prepare*2 message sent by the client. If so, it will send *Pre_prepare*3 message to other nodes of this group. The *Pre_prepare*3 message format is:

$$< Pre_prepare3, < h, v, S(p), bs, block >, hi > \quad (14)$$

Prepare phase **:** other nodes determines whether the output result of hi through the hash function is equal to ho or not. If so, they send a Supported message to the collection node of this group. Otherwise, they send Unsupported message to collection node of this group. The primary node acts as the collection node of this group. The *Supported* message format is:

$$< Supported, < h, v, S(p), bs, block >, S > \quad (15)$$

where S is the signature of the node by the private key. The *Unsupported* message format is:

$$< Unsupported, < h, v, S(p), bs, block >, S > \quad (16)$$

The collection node collects the Supported and Unsupported messages sent by the nodes of this group within a period of time. Then it sends the *Prepare* message to the primary node and other nodes in the group. The *Prepare* message format is:

$$< Prepare, < h, v, S(p), bs, block >, Ss > \tag{17}$$

where Ss is the sum of S in this group.

Commit phase **:** the primary node obtains the result through the counting votes. If the result is $f + 1$ supported votes, the primary node will send Commit message to other nodes, where f is the number of unsupported votes. The Commit message format is:

$$< Commit, < h, v, S(p), bs, block >, Sss > \tag{18}$$

where Sss is the sum of Ss with different collection nodes.

Reply phase **:** other nodes verify the message after they got the *Commit* message from the primary node. If it has $f + 1$ supported votes, they will write the new block in the local ledger, and then reply to the client.

V. PERFORMANCE ANALYSIS

A. SAFETY

During the JG-GHBFT *Pre_prepare* phase, if the client is a Byzantine node, it will send an error message to the verification node. The verification node then checks the message for correctness. If the received message does not match the message sent by the primary node, the verification node will broadcast an error signal to other verification nodes. The client request will not be processed when all verification nodes receive $x - 1$ error messages, where x is the number of groups.

In the JG-GHBFT *Prepare* phase, the collection nodes collect votes and send the voting results to primary node and the nodes in this group. In the JG-GHBFT *Commit* phase, the primary node will send the voting results of all groups to other nodes. After receiving the voting results, they will verify whether their votes have been tampered with. If tampering is detected, the node will send an error signal to the supervision node in this group. The error function nodes will be replaced when the supervision node receives more than half of the error signals. Regardless of whether the function nodes or clients are malicious, the consensus protocol can guarantee the security of the entire blockchain network.

B. FAULT TOLERANCE

Suppose N is the total number of nodes in the consensus network. The Byzantine fault tolerance of HC-PBFT [25] is (N/2 - 2x/3). JR-GHBFT consensus protocol can improve fault tolerance. The fault tolerance of the developed consensus network depends on the number of votes cast by the consensus node. Analysis of the developed protocol is as follows.

In the *Prepare* phase, the primary node collects the voting results of each group. The total number of nodes participating in the voting is SN. The total number of supported vote nodes is ST. The total number of unsupported vote nodes is SF. The relationship between them is (19). In order to reach consensus, it is necessary to satisfy inequality (20) that the number of supported votes exceeds the number of unsupported votes. Inequality (21) is derived from (19) and inequality (20) as follows:

$$SN = ST + SF + 1 \tag{19}$$

$$ST \geq SF + 1 \tag{20}$$

$$SF \leq \frac{SN - 2}{2} \tag{21}$$

We consider that downtime nodes cannot participate in voting in the JR-GHBFT Prepare phase. The total number of network nodes is N. The total number of non-voted nodes is O. Their relationship to SN is as following:

$$N = SN + O \tag{22}$$

In a large network, the total number of nodes participating in voting is far greater than the total number of nodes not participating in voting. Therefore, N is approximately equal to SN. As a result, the maximum fault tolerance of JR-GHBFT is $(N - 2)/2$. We can conclude that JR-GHBFT is superior to HC-PBFT [25] for fault tolerance when x is greater than 2.

C. EFFICIENCY

Communication complexity is a key indicator of consensus efficiency. Communication times can be used as an indicator to assess communication complexity. The total communication time of PBFT [17] after reaching a consensus is:

$$C_{PBFT} = 2N^2 - 2N \tag{23}$$

The total communication time of JG-GHBFT after reaching a consensus is:

$$C_{JR-GHBFT} = 2N + 3xy - 3x - 2 \tag{24}$$

where y is the number of nodes in each group. We consider that the relationship between N and x is $x = N/y$. Therefore, the total communication time of JG-GHBFT is $5N - 3x - 2$.

One can find from (24) that JR-GHPBFT reduces the communication complexity of PBFT [17] from $O(N^2)$ to $O(N)$.

D. WORKLOAD

When multiple nodes possess the same load capacity, the lower the number of requests they process from the same number of nodes at the same stage, the better the consensus structure. In blockchain systems, the primary node often handles a substantial number of requests during the *Prepare* phase, so we primarily focus on the workload condition of the node during this phase.

In the *Prepare* phase, the primary node workload of SBFT [35] W_{SBFT} is $N - 1$, while that of JR-GHBFT $W_{JR-GHBFT}$ is $x + y - 3$. The workload ratio of these two methods is:

$$PWR = \frac{W_{JR-GHBFT}}{W_{SBFT}} = \frac{x + y - 3}{N - 1} \tag{25}$$

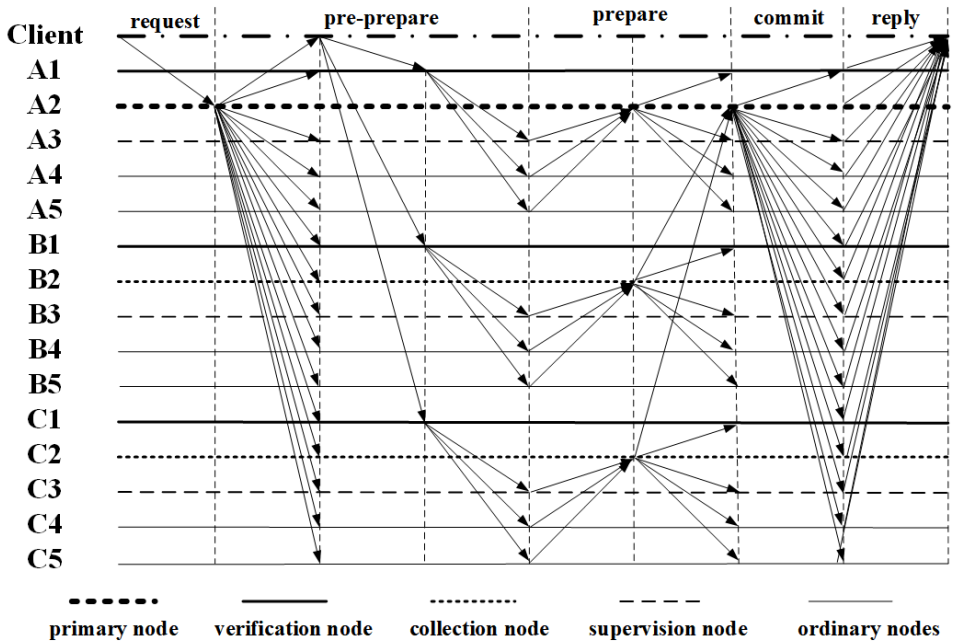


FIGURE 2. JR-GHBFT consensus protocol.

Given that the relationship between N and y is $y = N/x$, the PWR is:

$$PWR = \frac{x + \frac{N}{x} - 3}{N - 1} \tag{26}$$

We consider that N is held constant, the PWR value is minimized when $x = N/x$. We can get that the relationship between the grouping numbers and the total of network node ones as:

$$x = \sqrt{N} \tag{27}$$

We call the relationship in (27) as an adaptive grouping strategy.

It can be found in (26) that PWR tends to decrease gradually and its value is always less than 1 with the increase of N. The reason is that JR-GHBFT collection nodes carry most of the workload of the primary node.

VI. EXPERIMENTAL RESULTS AND DISCUSSIONS

In order to evaluate performance of the developed JR-GHBFT, we constructed an experimental network. In this network, all nodes run on one machine. Each node is equipped with a reputation value ledger and a ledger for recording transactions in the consensus process. This network and related models are deployed on a Linux system with an 8-core Intel i7-9200U CPU clocked at 3.6GHz and 32GB of RAM.

A. REPUTATION MODEL PARAMETERS

1) PARAMETER μ

Joint reputation is a key metric for assessing node reliability, which is composed of performance reputation and behavior reputation. The higher the joint reputation value for a node, the better its reliability. To obtain the best joint reputation value, the parameter $/mu$ in equation (11) is adjusted from 0.1 to 0.9 in increments of 0.2. Fig. 3 shows some experimental results with different μ value.

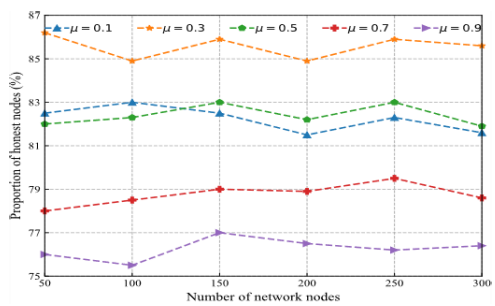


FIGURE 3. The proportion of normal nodes with different μ in (11).

It can be found in Fig. 3 that the proportion of normal nodes is the highest when μ is 0.3. Therefore, μ is set to 0.3 and remains the same in subsequent experiment.

2) PARAMETER p

Equation (28) has been introduced as evaluation index to evaluate the impact of selecting functional nodes from different proportions of candidate nodes on consensus network systems. It is represented as follows.

$$C = \sum_{i=1}^{N} (\alpha \times R_i + (1 - \alpha) \times F_i) \tag{28}$$

where R_i represents the number of candidate nodes reselected as functional nodes in the ith consistency process among $p \times N$ candidate nodes. F_i represents the number of Byzantine nodes selected as functional nodes in the ith consistency process among $p \times N$ candidate nodes. C serves as a quantitative

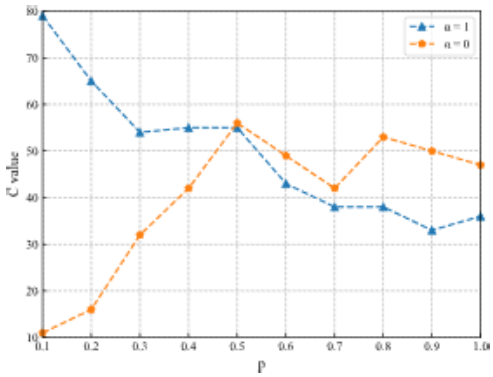


FIGURE 4. *C* value in (28) with different *p* values.

indicator measuring the centrality or security of consensus network systems, while α functions as a selector with a value of 0 or 1.

When α is 1, a larger value of C indicates a higher degree of centralization in the consensus network system. When α is 0, a larger value of C indicates a lower level of security in the consensus network system.

In the experiment, the value of N is set to 100, with 49 Byzantine nodes and 51 normal nodes. It is worth noting that half of the total network nodes are characterized by superior performance. The reason for setting the number of Byzantine nodes to 49 is based on the fault tolerance analysis in part B of Section V. It means that the consensus network system can tolerate up to 49 Byzantine nodes in this particular case. In addition, we would implement a reward and punishment system that applies to all network nodes after each round of consensus.

To get a reasonable p value, we changed it from 0.1 to 1 with an interval of 0.1. For each p, 100 rounds of consensus experiments have been done. Some results are shown at Fig. 4 to illustrate the influence of different p on the consensus network system.

One can find from Fig. 4 that the value of C decreases with the increase of p when α is 1 and p is less than 0.9. When p is greater than 0.9, the value of C shows an fluctuates trend. The value of C fluctuates with the increase of p when α is 0 and p is greater than 0.5. When p is less than 0.5, the value of C shows an upward trend. The centralization and security of the consensus network have roughly the same impact on the system when p is 0.5. The p is set to 0.5 at the experiment and keep it the same.

B. MODEL ANALYSES

1) PROBABILITY ANALYSIS

In a consensus network, if both the primary node and the verification node are Byzantine nodes, it will result in a consensus failure. We need to do a probabilistic analysis of this situation. Suppose that each node is independent of each other, we can get the probability of consensus failure as follows:

$$P = \frac{C_F^{x+1}}{C_N^{x+1}} \tag{29}$$

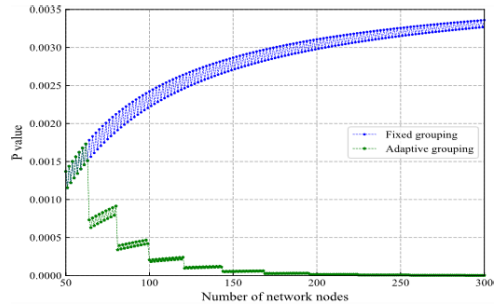


FIGURE 5. *P* value in (29) with different grouping strategies.

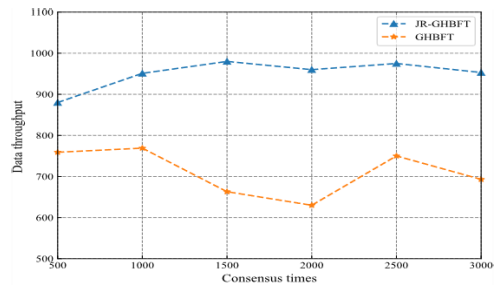


FIGURE 6. Data throughput with the joint reputation or not.

where F indicates the total number of Byzantine nodes in the network. C_F^{x+1} indicates that the $x+1$ nodes are randomly selected from F nodes when both the primary node and the verification node are Byzantine node. C_N^{x+1} indicates that the $x+1$ nodes are randomly selected from N consensus nodes when both the primary node and the verification node are Byzantine nodes.

Since the fault tolerance of JR-GHBFT is $(N-2)/2$ discussed in part B of section V, the range of F is $[0, (N-2)/2]$. When x is held constant, an increase of F results in a corresponding increase of P. The reason is that as the number of malicious nodes within a blockchain network increases, the probability of malicious nodes selected as function nodes increases.

We take $F = (N-2)/2$, and the fixed grouping number is set 7 for the sake of discussion. The Fig. 5 shows some experimental results with different grouping strategies. It can be found in Fig. 5 that the performance of adaptive grouping is better than that of fixed grouping. The reason is that the larger the number of groups there are, the lower the probability of malicious nodes gathering in the adaptive grouping strategy.

2) ABLATION EXPERIMENT

Data throughput is an important indicator for measuring the performance of consensus protocols. The higher the data throughput, the better the consensus protocol performance. The throughput refers to the number of data processed within a given time interval, typically measured in units such as bytes per second or packets per second. It is defined in the paper as follows:

$$TPS = \frac{transactions}{\Delta t} \tag{30}$$

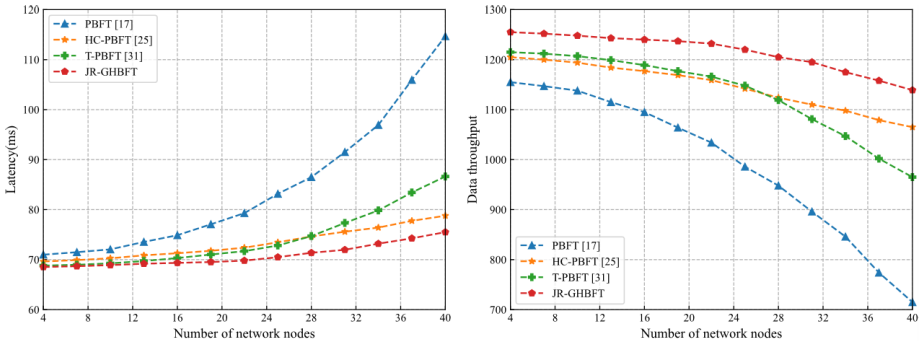


FIGURE 7. Comparisons of latency and data throughput.

TABLE 1. Comparisons with different consensus protocols.

Consensus protocols	PBFT [17]	HC-PBFT [25]	T-PBFT [31]	***JR − GHBFT***
Communication complexity	$O(N^2)$	$O(N)$	$O(N^2)$	$\boldsymbol{O(N)}$
Fault tolerance	$(N-1)/3$	$\geq (N-1)/3$	$\geq (N-1)/3$	$\boldsymbol{(N-2)/2}$
Scalability	Low	Medium	Medium	***High***
Degree of decentralization	High	High	Low	***High***

To test the JR-GHBFT, the number of network nodes is set to 50. There are 39 normal nodes, 10 Byzantine nodes and one client. The Fig. 6 shows some experimental results with joint reputation or not.

It can be found in Fig. 6 that JR-GHBFT is superior GHBFT for data throughput. The reason is that JR-GHBFT has chosen nodes with high joint reputation as the upper layer nodes.

C. COMPARISON WITH OTHER CONSENSUS PROTOCOLS

Some protocols including PBFT [17], HC-PBFT [25], and T-PBFT [31] are chosen to further evaluate the developed JR-GHBFT performance in data throughput and latency. For a fair comparison, we set the number of transactions to 2500. In the experiment, the number of network nodes increased from 4 to 40 in increments of 3. The Fig. 7 shows some results of data throughput and latency from left to right, respectively.

It can be found in Fig. 7 that the performance of developed JR-GHBFT is the best among the investigated protocols [17], [25], [31] for both latency and data throughput. Some of the reasons are as follows. Direct communication between a large number of nodes in PBFT [17] results in high communication complexity and low data throughput. HC-PBFT [25] reduces direct communication between a large number of nodes through grouping and hierarchical technology, which reduces communication complexity. However, since the primary node of each group is randomly selected, malicious nodes may be selected as primary nodes in HC-PBFT [25], which can reduce consensus efficiency to some extent. T-PBFT [31] selects nodes based on reputation values though, its communication complexity is still $O(N^2)$. Some reliable nodes can be selected as functional ones in the developed JR-GHBFT, which improves consensus efficiency. Besides JR-GHBFT reduces the communication complexity to $O(N)$ by improving the data transmission process.

In order to achieve greater fairness in results, certain additional indicators are considered. Results depend on the optimal parameters specified by the authors in their established protocol [17], [25], [31]. Table 1 shows a selection of comparison results.

It can be found in Table 1 that JR-GHBFT has the best performance among the investigated protocols [17], [25], [31]. Some of the reasons are as follows. Due to the communication complexity of $O(N^2)$, PBFT [17] has low efficiency in large networks. In HC-PBFT [25], it is difficult for lower layer nodes to verify the authenticity of messages delivered by upper layer nodes. In T-PBFT [31], the consensus group is only made up of highly trusted value nodes. The JR-GHBFT can reduce both communication complexity and centralization. In addition, the developed protocol can also tolerate more malicious nodes and improve scalability.

VII. CONCLUSION

A joint reputation model has been proposed to improve node reliability. The model can assess node credibility based on its behavior and performance. And it could also be applied to differentiate between normal and Byzantine nodes. Joint reputation based grouping and hierarchical model has been proposed to improve consensus efficiency. It is composed of both grouping and hierarchical models. The grouping model uses joint reputation values to balance node grouping so as to minimize overall differences in joint reputation values among groups. Some nodes with a joint reputation ranking in the top 50% of this group are randomly selected as function nodes, which improves the degree of decentralization. In the hierarchical model, nodes in each group are layered and the supervision nodes are mapped to the upper layer, which improves consensus network security and efficiency. Moreover, a new communication structure is designed to improve fault tolerance and reduce communication complexity by improving the three phases of PBFT [17]. Comparative experiments have shown the superiority of the developed protocol over other existing protocols.

We would further refine the consensus protocol in future research. The Preliminary Preparation phase can be streamlined to speed up the consensus process for the blockchain network. In addition, we plan to explore the application of blockchain technology for the protection of cultural relics, which facilitates the digitization of cultural relics.

REFERENCES

[1] M. Gimenez-Aguilar, J. M. de Fuentes, L. Gonzalez-Manzano, and D. Arroyo, "Achieving cybersecurity in blockchain-based systems: A survey," *Future Gener. Comput. Syst.*, vol. 124, pp. 91–118, Nov. 2021.

[2] M. Mostafa, "Bitcoin's blockchain peer-to-peer network security attacks and countermeasures," *Indian J. Sci. Technol.*, vol. 13, no. 7, pp. 767–786, Feb. 2020.

[3] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 6, pp. 4177–4186, Jun. 2020.

[4] S. Kaur, S. Chaturvedi, A. Sharma, and J. Kar, "A research survey on applications of consensus protocols in blockchain," *Secur. Commun. Netw.*, vol. 2021, pp. 1–22, Jan. 2021.

[5] D. Berdik, S. Otoum, N. Schmidt, D. Porter, and Y. Jararweh, "A survey on blockchain for information systems management and security," *Inf. Process. Manage.*, vol. 58, no. 1, Jan. 2021, Art. no. 102397.

[6] T. Meng, Y. Zhao, K. Wolter, and C.-Z. Xu, "On consortium blockchain consistency: A queueing network model approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 6, pp. 1369–1382, Jun. 2021.

[7] C. Tang, L. Wu, G. Wen, and Z. Zheng, "Incentivizing honest mining in blockchain networks: A reputation approach," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 1, pp. 117–121, Jan. 2020.

[8] M. Jakobsson and A. Juels, "Proofs of work and bread pudding protocols," in *Secure Information Networks*. Leuven, Belgium: Springer, Sep. 1999, pp. 258–272.

[9] B. David, P. Gaži, A. Kiayias, and A. Russell, "Ouroboros Praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain," in *Proc. 37th Annu. Int. Conf. Theory Appl. Cryptograph. Techn. Adv. Cryptol. (EUROCRYPT)*. Tel Aviv, Israel: Springer, May 2018, pp. 66–98.

[10] B. Wang, Z. Li, and H. Li, "Hybrid consensus algorithm based on modified proof-of-probability and DPoS," *Future Internet*, vol. 12, no. 8, p. 122, Jul. 2020.

[11] G. Sun, M. Dai, J. Sun, and H. Yu, "Voting-based decentralized consensus design for improving the efficiency and security of consortium blockchain," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6257–6272, Apr. 2021.

[12] R. Qiao, X.-Y. Luo, S.-F. Zhu, A.-D. Liu, X.-Q. Yan, and Q.-X. Wang, "Dynamic autonomous cross consortium chain mechanism in e-healthcare," *IEEE J. Biomed. Health Informat.*, vol. 24, no. 8, pp. 2157–2168, Aug. 2020.

[13] S. Banerjee, B. Bera, A. K. Das, S. Chattopadhyay, M. K. Khan, and J. J. P. C. Rodrigues, "Private blockchain-envisioned multi-authority CP-ABE-based user access control scheme in IIoT," *Comput. Commun.*, vol. 169, pp. 99–113, Mar. 2021.

[14] S. Pahlajani, A. Kshirsagar, and V. Pachghare, "Survey on private blockchain consensus algorithms," in *Proc. 1st Int. Conf. Innov. Inf. Commun. Technol. (ICIICT)*, Apr. 2019, pp. 1–6.

[15] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A survey of distributed consensus protocols for blockchain networks," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 2, pp. 1432–1465, 2nd Quart., 2020.

[16] L. S. Sankar, M. Sindhu, and M. Sethumadhavan, "Survey of consensus protocols on blockchain applications," in *Proc. 4th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, Jan. 2017, pp. 1–5.

[17] M. Castro and B. Liskov, "Practical Byzantine fault tolerance and proactive recovery," *ACM Trans. Comput. Syst.*, vol. 20, no. 4, pp. 398–461, Nov. 2002.

[18] Z. Zheng, S. Xie, H.-N. Dai, X. Chen, and H. Wang, "Blockchain challenges and opportunities: A survey," *Int. J. Web Grid Services*, vol. 14, no. 4, pp. 352–375, 2018.

[19] H. Sukhwani, J. M. Martínez, X. Chang, K. S. Trivedi, and A. Rindos, "Performance modeling of PBFT consensus process for permissioned blockchain network (Hyperledger Fabric)," in *Proc. IEEE 36th Symp. Reliable Distrib. Syst. (SRDS)*, Sep. 2017, pp. 253–255.

[20] O. Onireti, L. Zhang, and M. A. Imran, "On the viable area of wireless practical Byzantine fault tolerance (PBFT) blockchain networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.

[21] W. Lv, X. Zhou, and Z. Yuan, "Design of tree topology based Byzantine fault tolerance system," *J. Commun.*, vol. 38, no. Z2, pp. 143–150, 2017.

[22] W. Li, C. Feng, L. Zhang, H. Xu, B. Cao, and M. A. Imran, "A scalable multi-layer PBFT consensus for blockchain," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1146–1160, May 2021.

[23] T. Crain, V. Gramoli, M. Larrea, and M. Raynal, "DBFT: Efficient leaderless Byzantine consensus and its application to blockchains," in *Proc. IEEE 17th Int. Symp. Netw. Comput. Appl. (NCA)*, Nov. 2018, pp. 1–8.

[24] I. M. Coelho, V. N. Coelho, R. P. Araujo, W. Y. Qiang, and B. D. Rhodes, "Challenges of PBFT-inspired consensus for blockchain and enhancements over Neo dBFT," *Future Internet*, vol. 12, no. 8, p. 129, Jul. 2020.

[25] W. Zhong, X. Zheng, W. Feng, M. Huang, and S. Feng, "Improve PBFT based on hash ring," *Wireless Commun. Mobile Comput.*, vol. 2021, pp. 1–9, Nov. 2021.

[26] L. Chenzh, "Improved PBFT consensus mechanism based on kg medoids," *Comput. Sci.*, vol. 46, no. 12, 2019, Art. no. 101G107.

[27] H. Yoo, J. Yim, and S. Kim, "The blockchain for domain based static sharding," in *Proc. 17th IEEE Int. Conf. Trust, Secur. Privacy Comput. Communications/12th IEEE Int. Conf. Big Data Sci. Eng. (TrustCom/BigDataSE)*, Aug. 2018, pp. 1689–1692.

[28] N. Gao, Z. Chuangming, C. Yang, S. Lina, and W. He, "Improvement of PBFT algorithm based on network self-clustering," *Comput. Appl. Res.*, vol. 38, no. 11, pp. 3236–3242, 2021.

[29] J. Yang, Z. Jia, R. Su, X. Wu, and J. Qin, "Improved fault-tolerant consensus based on the PBFT algorithm," *IEEE Access*, vol. 10, pp. 30274–30283, 2022.

[30] G. Xu, H. Bai, J. Xing, T. Luo, N. N. Xiong, X. Cheng, S. Liu, and X. Zheng, "SG-PBFT: A secure and highly efficient distributed blockchain PBFT consensus algorithm for intelligent Internet of Vehicles," *J. Parallel Distrib. Comput.*, vol. 164, pp. 1–11, Jun. 2022.

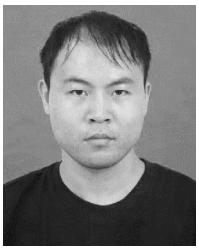
[31] S. Gao, T. Yu, J. Zhu, and W. Cai, "T-PBFT: An EigenTrust-based practical Byzantine fault tolerance consensus algorithm," *China Commun.*, vol. 16, no. 12, pp. 111–123, Dec. 2019.

[32] X. Yuan, F. Luo, M. Z. Haider, Z. Chen, and Y. Li, "Efficient Byzantine consensus mechanism based on reputation in IoT blockchain," *Wireless Commun. Mobile Comput.*, vol. 2021, pp. 1–14, May 2021.

[33] W. Liu, X. Zhang, W. Feng, M. Huang, and Y. Xu, "Optimization of PBFT algorithm based on QoS-aware trust service evaluation," *Sensors*, vol. 22, no. 12, p. 4590, Jun. 2022.

[34] P. Boldi, F. Bonchi, C. Castillo, and S. Vigna, "Voting in social networks," in *Proc. 18th ACM Conf. Inf. Knowl. Manage.*, Nov. 2009, pp. 777–786.

[35] G. G. Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. K. Reiter, D.-A. Seredinschi, O. Tamir, and A. Tomescu, "SBFT: A scalable and decentralized trust infrastructure," 2018, *arXiv:1804.01626*.



HAO QIN is currently pursuing the M.S. degree in signal and information processing with the School of Communication and Information Engineering, Shanghai University, Shanghai, China. His current research interests include security and privacy in blockchain technology.

YEPENG GUAN received the B.S. and M.S. degrees in physical geography and the Ph.D. degree in geo-detection and information technology from Central South University, Changsha, China, in 1990, 1996, and 2000, respectively. He is currently a Full Professor with the School of Communication and Information Engineering, Shanghai University, China. His current research interests include machine learning, cloud computing, and blockchain.